

gif-server — Deployment Guide

Self-hosted deployment to a Docker homeserver behind a containerized nginx reverse proxy. Generated 2026-05-23.

1. Architecture

Three pieces, all on the homeserver:

- **db** — Postgres 16 container (gif-server's compose stack).
- **app** — the gif-server binary container. Migrations run automatically on startup. Bound to `127.0.0.1:8847` on the host (for debugging) and joined to the existing `nginx` Docker network with alias `gif-server`.
- **nginx** — a pre-existing container (`lscr.io/linuxserver/nginx`) on the `nginx` network. The sole public ingress; terminates TLS and proxies to `http://gif-server:8847`.

The app is never published to the internet. The internet reaches it only through nginx.

2. Prerequisites

- Docker + Docker Compose on the homeserver (already present).
- The existing `nginx` Docker network and nginx container.
- DNS A/AAAA record for the public domain (e.g. `gifs.example.com`) pointing at the homeserver's public IP.
- Router port-forward of TCP **80** and **443** to the homeserver.
- A TLS certificate for the domain, mounted into the nginx container (handled separately).
- A few GB of free RAM/disk — the first Rust build is slow and memory-hungry.

3. Repository files involved

File	Role
<code>docker-compose.yml</code>	db + app stack. Config comes from <code>.env</code> via <code>\${VAR}</code> interpolation. App

joins the external `nginx` network.

`.env.example`

Template for configuration. The real `.env` is created on the server and is git-ignored / not rsynced.

`deploy/
nginx.conf.example`

Reverse-proxy server block to copy into the `nginx` container's site-confs.

`Dockerfile`

Multi-stage Rust build (cargo-chef) producing a slim runtime image.

4. Step 1 — Copy the code to the server

From the development machine, in the project directory:

```
rsync -avz --delete \  
  --exclude target/ --exclude .git/ --exclude storage/ --exclude .env \  
  ./ user@homeserver:/srv/gif-server/
```

Excludes the build cache, git history, server-side GIF data, and the `env` file. Run once with `-n` (dry run) first to preview, then without.

First time: ensure the target exists and is writable by your SSH user:

```
sudo mkdir -p /srv/gif-server  
  
sudo chown "$USER:$USER" /srv/gif-server
```

5. Step 2 — Create the production `.env` on the server

At `/srv/gif-server/.env` (it is not rsynced, so create it fresh). Generate secrets with `openssl rand`:

```
POSTGRES_PASSWORD=<openssl rand -hex 24>  
SESSION_SECRET=<openssl rand -hex 32>  
BASE_URL=https://gifs.example.com  
MATRIX_SERVER_NAME=matrix.example.com  
MATRIX_FEDERATION_URL=https://matrix.example.com  
MATRIX_ADMIN_MXIDS=@admin:matrix.example.com
```

```
CORS_ALLOWED_ORIGINS=https://<your-frontend-origin>
```

- `DATABASE_URL` is **not** needed here — compose builds it from `POSTGRES_PASSWORD` with host db.
- `SESSION_SECRET` must be at least 32 characters.
- `BASE_URL` must be the public `https://` URL — it is embedded in rendition URLs.
- `STORAGE_MAX_BYTES` / `PER_USER_STORAGE_BYTES` fall back to defaults (10GB / 1GB) if omitted.

Create `.env` **before** the first `docker compose up`. Required variables use `${VAR:?}` markers, so compose fails loudly (by design) if any are missing.

6. Step 3 — Build and start the stack

```
cd /srv/gif-server
docker compose up -d --build
docker compose logs -f app          # watch migrations apply + startup
curl -fsS http://127.0.0.1:8847/health # expect HTTP 200 from the host
```

A clean boot logs the migrations (`001_init.sql`, `002_hidden.sql`) then "listening on 0.0.0.0:8847".

7. Step 4 — Add the nginx site

Copy `deploy/nginx.conf.example` into the nginx container's site-confs directory on the host:

```
cp /srv/gif-server/deploy/nginx.conf.example \
  /docker/apps/nginx/nginx/site-confs/gif-server.conf
```

Edit it: set the real `server_name` and the container-internal TLS cert paths. Key points already baked into the template:

- Proxies to `http://gif-server:8847` using Docker's embedded DNS resolver (`127.0.0.11`) and a variable upstream, so an app restart doesn't strand nginx on a stale IP.
- `client_max_body_size 50m;` — matches the app's 50 MB upload limit (nginx defaults to 1 MB).
- Overwrites `X-Forwarded-For` / `X-Real-IP` with `$remote_addr`.

Then validate and reload nginx:

```
docker exec <nginx-container> nginx -t
docker exec <nginx-container> nginx -s reload
curl -fsS https://gifs.example.com/health # expect HTTP 200 over TLS
```

8. Step 5 — Verify

- **App resolvable from nginx:** `docker exec <nginx-container> getent hosts gif-server` returns an IP.
- **Real client IPs:** nginx access logs show public client IPs, not `172.x` gateway addresses — confirms the per-IP rate limiter keys on real clients.
- **Auth path:** `POST /auth/matrix` with a real Matrix OpenID token returns a JWT; bad tokens return 401 (not 500).

Security — X-Forwarded-For. The auth rate limiter trusts the leftmost X-Forwarded-For value. nginx must **overwrite** it with `$remote_addr` (the template does), **not** append via `$proxy_add_x_forwarded_for` — appending would let a client prepend a spoofed IP and bypass the per-IP login limiter. The app must remain reachable only through nginx (it is: localhost bind + internal network, never publicly published). A non-spoofable global auth cap (~5 req/s) bounds total login load as a second layer.

9. Local development (unchanged)

Local dev runs the app on the host against the dockerized db:

```
./scripts/dev.sh # starts db via compose, waits healthy, runs cargo run
# or, if db is already up:
cargo run
curl -i http://localhost:8847/health
```

- The app reads the local `.env` (localhost DB URL, localhost base URL).
- `docker compose up -d db` works locally despite the external `nginx` network (only the app service references it).
- Running the **full** stack locally (`docker compose up`) requires the network to exist: `docker network create nginx` first.

10. Routine operations

Task	Command
Redeploy after code change	rsync (Step 1), then <code>docker compose up -d --build</code>
View app logs	<code>docker compose logs -f app</code>
Restart app only	<code>docker compose restart app</code>
Stop everything	<code>docker compose down</code> (data persists in the <code>postgres_data</code> volume)

GIF files live in the host bind mount `/srv/gif-server/storage` and are root-owned (the container runs as root). Back up that directory plus the Postgres volume.